

## Laporan Ujian Akhir Semester



**Mochammad Reezqi Pratama - 11231041**  
**Sistem Paralel dan Terdistribusi A**

### Ringkasan Sistem dan Arsitektur

Sistem yang dibangun adalah sebuah Pub-Sub Log Aggregator terdistribusi berbasis microservices menggunakan Docker Compose. Arsitekturnya terdiri dari Aggregator sebagai API FastAPI untuk menerima/menyimpan event, Publisher sebagai generator event simulasi yang mencakup retries atau duplicates, Message Broker untuk buffering dan decoupled messaging, Database PostgreSQL sebagai durable storage untuk event store.

Arsitektur yang digunakan adalah sebagai berikut;

- Idempotency & Dedup: Digunakan pendekatan unique constraint kombinasi kolom ("topic", "event\_id") pada PostgreSQL. Ketika worker memproses event yang sama, PostgreSQL akan menolak duplikasi tanpa menyebabkan error di sistem menggunakan "ON CONFLICT DO NOTHING". Jadi walaupun banyak worker yang bekerja sekaligus tetap aman.
- Transaksi & Konkurensi: Isolation level yang digunakan adalah "READ COMMITTED", dikombinasikan dengan upsert/unique constraint dan instruksi SQL update. Efektif dalam mencegah lost-update karena instruksi update dievaluasi ulang terhadap data committed terbaru dan row-level locking diterapkan saat write.
- Ordering: Menggunakan metadata timestamp pada level event. Saat di query "GET /events", data diurutkan ulang order by timestamp DESC berdasarkan waktu pembuatan.

### **T1: Karakteristik Sistem Terdistribusi dan Trade-off Desain**

Sistem terdistribusi memiliki karakteristik konkurensi antar komponen, tidak adanya sistem jam waktu global yang utuh, dan kegagalan independen di setiap node (Coulouris et al., 2011). Pada rancangan Pub-Sub Agregator, trade off yang diambil adalah mengutamakan reliabilitas dan skalabilitas dengan menambahkan Redis sebagai broker. Konsekuensinya adalah adanya latency tambahan antara publikasi event dan persisten event ke basis data dibandingkan dengan direct insert.

### **T2: Pilihan Arsitektur Publish-Subscribe vs Client-Server**

Arsitektur publish-subscribe menjamin decoupling antara publisher dan subscriber, yang membebaskan sistem dari ketergantungan temporal dan referensial secara point-to-point (Coulouris et al., 2011). Pendekatan ini cocok diterapkan pada kasus Log Agregator karena berbagai sumber (mobile app, API eksternal) tidak perlu menunggu pemrosesan penyimpanan selesai untuk melanjutkan proses utama mereka. Model client-server tradisional akan lebih sinkron terhadap asimetri beban logging.

### **T3: At-least-once vs Exactly-once Delivery dan Peran Idempotent Consumer**

Dalam komunikasi network, pengiriman exactly-once sangat kompleks karena anomali jaringan bisa memicu re-transmisi otomatis (Coulouris et al., 2011). Karena itulah model at-least-once delivery umumnya lebih sering dipakai. Akibatnya, penerima akan berpotensi memproses duplikasi data. Di sinilah peran dari idempotent consumer. Dirancang agar eksekusi berulang terhadap payload atau event yang sama tidak merubah representasi internal, sehingga hasil akhirnya setara dengan exactly-once.

### **T4: Skema Penamaan dan Event-ID**

Skema penamaan pada struktur pub-sub aggregator memanfaatkan gabungan properti topic dan pengenalan unik event\_id yang dihasilkan melalui mekanisme Universally Unique Identifier (UUID) dari sisi sumber atau sistem lokal pembuat pesan. Berdasarkan (Coulouris et al., 2011), penamaan nama lokal yang bisa dijadikan murni global (unik) mempermudah referensi ulang yang dijamin tidak tabrakan. Kombinasi topic yang mengklasifikasikan wilayah pesan serta “event\_id” memfasilitasi pembuatan constraint unik pada dedup store (PostgreSQL) yang menjamin efisiensi resolusi konflik duplikat data.

### **T5: Ordering Praktis**

Mengatur kesepakatan waktu secara universal adalah masalah paling umum dalam sistem terdistribusi. Pengurutan bisa diatur melalui pendekatan pragmatis menggunakan kombinasi cap waktu timestamp ISO 8601 dari sumber dan jika perlu monotonic counter (Coulouris et al., 2011). Dampak negatif dan batasan pendekatan timestamp sinkron ini adalah adanya clock skew di sistem asalnya, sehingga ordering antar-sumber tidak selalu akurat 100% pada hitungan milidetik. Namun, untuk sekedar pengelompokan riwayat aggregator events, akurasi ini dapat dimaklumi dan sangat efisien secara operasional karena tidak butuh komunikasi sinkronisasi berat seperti pada algoritma lamport clock.

## **T6: Failure Modes dan Mitigasi**

Failure mode atau kegagalan yang sering terjadi adalah kegagalan crash node, hilangnya jaringan secara acak, serta kelambanan (Coulouris et al., 2011). Mitigasi aggregator diterapkan melalui crash recovery yang difasilitasi oleh Redis queue yang menjaga buffer message belum terproses serta persistensi deduplikasi yang ada pada volume PostgreSQL. Jika container aggregator berhenti tiba-tiba, setelah di-restart, consumer dapat dengan aman melanjutkan antrian yang tersisa. Kemungkinan re-processing dicegah oleh PostgreSQL dedup store “ON CONFLICT DO NOTHING” di mana event yang status prosesnya parsial tersimpan pada proses lalu tidak akan menimbulkan data inkonsisten.

## **T7: Eventual Consistency dan Deduplication**

Konsep eventual consistency adalah kebebasan perbarui lokal di mana setelah serangkaian operasi di seluruh node, status akan mengarah sama meski sementara mungkin berbeda (Coulouris et al., 2011). Event pada node aggregator masuk ke antrian asinkron. API pengguna tidak segera melihat event yang dipublikasi melalui “events” secara real-time tapi ada delay wajar. Peran idempotency dan deduplication di fase sinkronisasi asinkron untuk memastikan ketika node sinkron dengan antrian, nilai gabungan di database store tetap konsisten meskipun worker konstan mengulang-ulang insersi data tabrakan dari berbagai jaringan retry logs.

## **T8: Desain Transaksi**

Transaksi dijamin atas dasar standar fungsi ACID: Atomicity, Consistency, Isolation, dan Durability (Coulouris et al., 2011). Skema insert event dan penghitungan jumlah pada aplikasi diisolasi. Mengatur batasan kehilangan pembaruan (lost-update) diimplementasikan dengan tidak melakukan siklus baca-dan-tulis (read-modify-write cycle), namun didelegasikan sebagai satu instruksi spesifik di engine DB (“UPDATE”), sehingga kunci internal memori database secara otentik menjaga mutually-exclusive updates di atas level baris yang sama.

## **T9: Kontrol Konkurensi**

Dalam arsitektur perangkat pesan majemuk ini, implementasi kontrol konkurensi diatasi melalui desain skema pesimistik di pangkalan data melalui unique constraints. Ketika instruksi “INSERT” berusaha dipenuhi oleh beberapa instansi pekerja secara bersamaan (multi-thread atau multi-pod worker) untuk tuple (topic, event\_id) yang sama, PostgreSQL menerapkan konvensi idempotent write pattern dengan sintaks “ON CONFLICT”. Hal ini secara transaksional melempar atau membatalkan insersi duplikat tanpa race condition, sebagaimana disyaratkan dalam (Coulouris et al., 2011) di mana server handle conflict resolution internal.

## **T10: Orkestrasi, Keamanan Jaringan dan Observability**

Menurut (Coulouris et al., 2011), keamanan pada level sistem berkas dan integrasi orkestrasi membutuhkan isolasi sumber daya. Hal ini diwujudkan pada Docker Compose melalui isolasi network lokal; port basis data (5432) dan broker (6379) tidak exposed secara publik melainkan hanya port layanan aggregator (8080) ke antarmuka client. Manajemen persisten di-mount ke dalam unit docker volume (“pg\_data”, dsb) di-detach dari container lifecycle mencegah kerusakan akibat restart operasional. Fitur analitik/telemetry atau observability dipenuhi sebagian lewat fungsional API metrik “/stats” yang terus merangkum riwayat serta standar ‘logging’ di stdout Python handler.

### **Analisis Performa dan Hasil Uji Konkurensi**

Berdasarkan metrik eksekusi “pytest” (tests/test\_api.py):

1. Konkurensi: Pada test “test\_concurrency\_race\_condition”, 10 thread yang dikirimkan secara bersamaan memposting payload ID unik dan seragam yang persis sama ke endpoint terpusat tidak menimbulkan data ganda pada tabel final. Data masuk secara tepat dengan 9 data dideklarasikan gagal (duplicate dropped) dan hanya 1 valid, membuktikan kontrol konkurensi bekerja presisi.
2. Stress dan Responsivitas: “test\_stress\_small\_batch” yang melakukan push batch array size berjumlah 500 iterasi mampu diolah dalam status respons HTTP di bawah rata-rata ~50ms berkat konsep delegasi proses penulisan basis data berat ke sinkronisasi asinkron Redis Queue oleh Fast API.

### **Daftar Pustaka**

*Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2011). \*Distributed systems: Concepts and design\* (5th ed.). Addison-Wesley.*

Link Github: <https://github.com/FhukoMai/UAS>